

Apple container → Containers submodule integration — DESIGN (research + design only)

Revision: 1 **Last modified:** 2026-06-13T00:00:00Z **Authority:** vasic-digital/tmux project; extends Containers/ (vasic-digital/Containers) per §11.4.74 (catalogue-first / extend-don't-reimplement) + §11.4.76 (Containers-submodule mandate) + §11.4.28 (owned-submodule decoupling) **Scope:** READ-ONLY research/design. No code edited, nothing installed, nothing committed during the producing task. **Goal:** On a macOS / Apple-Silicon host, create + run a containerized **Linux** environment on demand to build the tmux Linux binary and run the tmx (tmux) Linux test suite, without a separate Linux box — by EXTENDING the Containers submodule's existing crossbuild engine abstraction with an Apple-container backend.

PART A — Apple container latest-source research

Apple container is Apple's open-source CLI (apple/container, npm-irrelevant; distributed as a signed .pkg or via Homebrew) that runs **Linux** containers on macOS by booting **one lightweight virtual machine per container** through the **Virtualization framework**, with a minimal Linux guest userspace. It consumes and produces **standard OCI images**, so it can pull alpine / ubuntu / debian from Docker Hub and other OCI registries.

Requirements (load-bearing)

- **Apple Silicon Mac required** (arm64). No Intel-Mac support.
- **macOS version:** The README/installation docs state container "is supported on macOS 26, since it takes advantage of new features and enhancements to virtualization and networking." It **runs** on macOS 15 (Sequoia) but with documented limitations, and the maintainers state issues that cannot be reproduced on macOS 26 typically will not be fixed.
- **Administrator privileges** required for the one-time .pkg install (files land in /usr/local). This is the one-time bootstrap, NOT per-run.
- Runs unprivileged after install — no sudo for container run/exec. Matches the submodule's rootless-only / no-sudo mandate (§6.U).

Install method (one-time bootstrap — outside test execution per §11.4.98)

- Homebrew: `brew install --cask container (cask)`, or download the signed .pkg from apple/container GitHub Releases and double-click (admin password prompt).

- Files install under `/usr/local`; binary at `/usr/local/bin/container`.
- Update/downgrade: `/usr/local/bin/update-container.sh (-v 0.3.0 pins a version)`.
- Uninstall: `/usr/local/bin/uninstall-container.sh -k (keep data) / -d (remove data)`.
- **One-time system bring-up:** `container system start` (starts the launchd/XPC system service; offers to install a default Linux kernel on first run). This is the on-demand-infra entry point (the submodule's §11.4.76 "boot is part of the test entry point" rule maps `container system start` to `pkg/boot`).

CLI surface needed for our use case (verbatim flags from the official command reference)

- **System lifecycle:** `container system start`, `container system stop`, `container system status`.
- **Images:** `container image pull <ref>` (alias `container images pull`), `container image list` / `container image ls`.
- **Run:** `container run [flags] <image> [cmd...]`
 - `-d`, `--detach` — run detached.
 - `-i`, `--interactive` + `-t`, `--tty` — interactive/TTY.
 - `--rm`, `--remove` — remove container after it stops.
 - `--name <name>` — set the container ID/name.
 - **Mount host dir (CRITICAL):** `-v`, `--volume <volume>` — bind mount a volume; OR `--mount type=<>,source=<HOSTDIR>,target=<CONTAINERDIR>,readonly` — the explicit, unambiguous form the adapter will use for mounting the tmx repo.
 - `-c`, `--cpus <n>` — CPU count (resource limit).
 - `-m`, `--memory <size>` — memory with K/M/G/T/P suffix (resource limit; honors §12.6 60% ceiling).
 - `-e`, `--env key=value` — environment variables.
 - `-w`, `--workdir`, `--cwd <dir>` — initial working directory.
 - `--entrypoint <cmd>` — override entrypoint.
- **Exec (run a command in a running container + capture stdout):** `container exec [flags] <container> <cmd...>` — same process flags as `run`. The adapter runs `container exec <name> sh -c '<cmd>'`, captures stdout/stderr via `cmd.Stdout/cmd.Stderr` buffers, and reads the process exit code from `ProcessState.ExitCode()` (identical mechanism to the existing `realContainerRunner`).
- **List:** `container list` / `container ls (-a, --all includes stopped)`.
- **Stop / kill:** `container stop <name>` (graceful signal), `container kill <name>` (SIGKILL).
- **Remove:** `container delete <name>` / `container rm <name>`.
- **Build:** `container build -t <name> -f <Dockerfile> <context>` (Dockerfile-compatible).

Getting exact stdout + exit code

`container run`/`container exec` write the guest process's stdout/stderr to the host stdio streams; the host process exit code reflects the container command's exit code. The adapter therefore uses `Go exec.CommandContext` with separate `bytes.Buffer` for stdout/stderr and `ProcessState.ExitCode()` — exactly the pattern `pkg/crossbuild/container_runner.go` already uses for `podman/docker`.

OCI image compatibility

Yes. "Since container consumes and produces standard OCI images, you can easily build with and run images produced by other container applications." Standard alpine / ubuntu / debian pull from Docker Hub. arm64 image variants run natively (no emulation) on Apple Silicon, which is what we want for an arm64 Linux tmux test run.

Known limitations / gotchas (documented)

- **macOS 15 networking is severely limited:** all containers attach to the **default vmnet** network (default CIDR 192.168.64.1/24); **container-to-container communication over the virtual network is not possible**, and multiple networks are unsupported. Networking improvements are tied to macOS 26. **Impact on us: NONE** — the tmx test flow needs only host→container exec and a host-dir bind mount; it does not need container-to-container networking.
- **Rosetta / x86 emulation:** Rosetta-backed x86 Linux execution has known regressions with recent kernels (some Fedora/Debian images segfault under Rosetta + kernel 6.13). **Impact on us: avoid x86** — run the **arm64** Linux image natively; do not rely on Rosetta. This is correct anyway for an Apple-Silicon host running an arm64 Linux build of tmux.
- **First-boot latency:** each container run boots a fresh micro-VM; first boot of a new image incurs VM + kernel start latency (seconds). Tests must allow a startup window before exec.
- **Pre-1.0 stability:** breaking changes possible until v1.0.0; stability guaranteed only within patch versions. The adapter must not hard-pin flag spellings beyond what the command reference documents, and should degrade with an honest error if container is absent or the macOS version is unsupported.

Honest host blocker (this machine)

- This host is **macOS 15.5 (build 24F74)**, **arm64**, and container is **NOT installed** (command `-v container` → not found). Apple container is **officially supported on macOS 26**; it runs on macOS 15 with the limitations above, but our flow (host-dir mount + host→guest exec, no inter-container networking) sits entirely within what macOS 15 supports. The realistic blockers are: (1) one-time brew `install --cask container + container system start` (admin password once — operator action), (2) acceptance that on macOS 15 unreproducible-on-26 bugs may not be fixed upstream. If macOS 15 proves unstable for the kernel/image we pick, the §11.4.81 honest-gap path applies: SKIP-with-reason citing "macOS 26 required for full support" and fall back to the existing podman/docker LinuxContainerBackend (Podman Machine on Apple Silicon).

Sources verified 2026-06-13

- <https://github.com/apple/container> (README — requirements, install .pkg, container system start, OCI images, lightweight VMs)
- <https://github.com/apple/container/blob/main/docs/command-reference.md> (CLI commands + flags: run/exec/ls/stop/kill/delete/system/image/build, -v/--mount, -c/-m/-e/-w)

- <https://github.com/apple/container/blob/main/docs/technical-overview.md> (one-VM-per-container, Virtualization framework, OCI compatibility, macOS 15 vmnet networking limitations + default CIDR)
 - <https://github.com/apple/container/issues/76> (Rosetta-for-Linux status)
 - <https://www.outcoldman.com/blog/2026/05/02/apple-container-tired-of-docker/> (community usage report)
 - <https://chamodshehanka.medium.com/apples-new-containerization-framework-a-deep-dive-into-macos-s-future-for-developers-cf102643394a> (macOS 15 limitations, Rosetta/kernel-6.13 regressions, macOS 26 networking)
-

PART B — Containers submodule extension-point analysis (actual Go code)

Catalogue-Check: extend vasic-digital/Containers (pkg/crossbuild) — the engine abstraction already exists; we ADD a backend, we do NOT reimplement.

The existing "run a Linux container / pick an engine" abstraction

pkg/crossbuild/ is exactly the right seam. It is a Strategy + Selector design:

- **pkg/crossbuild/types.go** defines the strategy interface:
 - type Backend interface { Name() string; Capabilities() Capabilities; Build(ctx, BuildRequest) BuildResult }
 - BuildRequest{ Target{OS,Arch}, SourceDir, BuildCommand, OutputSubpath, HostOutputDir, Environment, Timeout }
 - BuildResult{ Target, ArtifactPath, ArtifactSize, BackendName, Duration, StdoutTail, StderrTail, Error }
 - Capabilities{ SupportsTargets []Target, RequiresHostOS []string, IsolatesEnvironment bool, ArtifactNotes string }
- **pkg/crossbuild/selector.go** is the single engine-selection decision point:
 - Selector.Register(b Backend) (first-registered wins on ties), Choose(req) matches req.Target against each backend's Capabilities.SupportsTargets AND RequiresHostOS against the host OS, Build(ctx, req) = Choose + Build. Host OS/arch comes from runtime.GOOS/GOARCH via NewSelector(); NewSelectorForHost(os,arch) is the test seam.
- **pkg/crossbuild/container_runner.go** is the shell-out seam the new backend reuses the *shape* of:
 - type containerRunner interface { ImageExists(ctx, ref) bool; Run(ctx, containerRunSpec) (exitCode int, err error) }
 - containerRunSpec{ Image, MountSource, MountTarget, WorkDir, Command, Environment, Stdout, Stderr }
 - realContainerRunner builds podman/docker run --rm -v src:target:Z -w wd <image> sh -c <cmd> via exec.CommandContext, captures stdout/stderr buffers, reads ProcessState.ExitCode().
 - pickContainerBinary() does command -v-style detection: exec.LookPath("podman") then "docker".
- **pkg/crossbuild/linux_container.go** — LinuxContainerBackend is the existing concrete backend: SupportsTargets: [{linux, <arch>}],

RequiresHostOS: nil (works on every host where podman/docker exist), IsolatesEnvironment: true, mounts req.SourceDir → /work/src, runs BuildCommand, asserts a non-zero artifact (anti-bluff). **This is the file the new Apple-container backend mirrors.**

- **pkg/crossbuild/host_direct.go** — HostDirectBackend (host-native fast path) shows the processRunner seam + realRunner + validateRequest + anti-bluff zero-byte-artifact guard reused by every backend.

How an engine is SELECTED

Selector.Choose (selector.go). It does NOT itself command `-v` — it matches declared Capabilities. The runtime availability check lives **inside** each backend's runner (ImageExists / the pickContainerBinary exec.LookPath). So registering Apple-container as a macOS-only engine = register a backend whose Capabilities.RequiresHostOS: ["darwin"] and whose runner detects /usr/local/bin/container (or exec.LookPath("container")). Because first-registered wins, registering the Apple-container backend **before** LinuxContainerBackend makes it the preferred Linux-on-macOS engine, with podman/docker as the documented fallback.

(Separately, pkg/runtime/detect.go holds the OTHER engine abstraction — ContainerRuntime with a RuntimePriority list podman → docker → nerdctl → cri-o → lxd → kubernetes and AutoDetect. That layer is for long-running service orchestration (compose/health/distribution), not cross-build. For the tmx Linux-test use case the **crossbuild** seam is the correct one; optionally "apple-container" can later be added to RuntimePriority if compose-style orchestration on macOS is ever needed — out of scope here.)

Existing test patterns the new backend must follow

- Backend tests inject a **fake runner** (fakeContainerRunner in linux_container_test.go via newLinuxContainerBackendWithRunner) and assert orchestration: happy-path (runCalled, ArtifactSize, command contains the build token), missing-image honest error, zero-byte-artifact-is-bluff, capabilities-honest, default-image-ref-by-arch.
- selector_test.go exercises NewSelectorForHost to pretend a different host OS and assert routing — the seam to prove "on darwin, the apple-container backend is Chosen for linux/arm64".
- Real-stack proof is a **Challenge** under Containers/challenges/scripts/ (per the submodule's "Challenge Coverage" mandate + §11.4.76 anti-bluff "must actually boot them via the Submodule"), plus a paired meta-test mutation per §1.1.

PART C — Integration design

C.1 The minimal Apple-container backend to ADD to the Containers submodule

- **Package path:** Containers/pkg/crossbuild/apple_container.go (+ apple_container_test.go). Same package, mirrors linux_container.go.

- **Interface implemented:** `crossbuild.Backend (Name/Capabilities/Build)`.
- **New struct:** `AppleContainerBackend` with an injected `appleContainerRunner` seam (mirrors `containerRunner`) so tests use a fake and production uses `realAppleContainerRunner`.
- **Capabilities (honest):**
 - `SupportsTargets:` `[{OS:"linux", Arch:"arm64"}]` (native Apple-Silicon Linux; intentionally NOT `amd64` — avoids the Rosetta/kernel-6.13 regression).
 - `RequiresHostOS:` `["darwin"]` (Apple container is macOS-only) — this is the honest restriction the Selector routes on.
 - `IsolatesEnvironment:` `true`,
 - `ArtifactNotes:` `"Linux arm64 build inside an Apple-container lightweight VM (Virtualization framework)"`.
- **Exact container CLI the runner shells out to** (via `exec.CommandContext`, `stdout/stderr` buffers, `ProcessState.ExitCode()`):
 - **Ensure system up (on-demand infra, §11.4.76):** container system status; if not running → container system start.
 - **Image present?** container image list (parse for the ref) — the `ImageExists` equivalent; or container image pull <ref> to provision.
 - **Run with host-dir mount + capture (the core call):** container run --rm --name <generated> --mount type=virtiofs,source=<SourceDir>,target=/work/src -w /work/src -c <cpus> -m <mem> [-e K=V ...] <imageRef> sh -c '<BuildCommand>' (If type=virtiofs is rejected by the installed version, fall back to -v <SourceDir>:/work/src. The adapter probes once and records which form worked — no guessing per §11.4.6.)
 - **Long-lived + exec variant (for multi-step build-then-test):** container run -d --name <n> --mount ... <image> sleep infinity; then container exec <n> sh -c '<cmd>' per step (build, then run test suite), capturing each step's stdout + exit code; then container stop <n> + container rm <n> in a deferred cleanup (§11.4.14).
 - **Resource limits:** -c/-m clamp to the §12.6 60%-memory ceiling computed from host RAM (reuse the host-safety budget helper pattern).
- **Anti-bluff guards reused from `linux_container.go`:** non-zero artifact assertion, zero-byte-is-bluff error, missing-image honest error pointing at provisioning docs + a SKIP-OK ticket, `validateRequest`.
- **Registration:** the consuming project (tmx) builds a Selector, registers `AppleContainerBackend` **before** `LinuxContainerBackend` and `HostDirectBackend`, so on a darwin host targeting linux/arm64 the Apple-container engine wins, with podman/docker as fallback when container is absent.

C.2 How the tmx project consumes it (per §11.4.76)

1. **Add the submodule + go.mod replace.** tmx already has `Containers/` as a submodule; consume `digital.vasic.containers` via a replace `digital.vasic.containers => ./Containers` directive during development + a pinned SHA for release (the §11.4.76 replace-directive rule). `helix-deps.yaml` already declares the dependency.
2. **Mount the tmx repo into a Linux container.** tmx code calls `crossbuild.NewSelector()`, registers the backends, and issues a `BuildRequest` with `SourceDir = <tmx repo root>`, mounted at `/work/src`.

3. **Build the tmux Linux binary inside.** BuildCommand runs tmx's Linux build (bash scripts/setup.sh --build-only or the equivalent in-container build, after installing build deps in the image) targeting linux/arm64; OutputSubpath points at the produced tmux binary; the backend asserts it is non-zero (anti-bluff) and copies it to HostOutputDir.
4. **Run the Linux test suite + capture results back.** Using the -d + exec variant: after build, container exec <n> sh -c 'bash scripts/tests/run_all.sh', capturing stdout/exit code into the BuildResult.StdoutTail / a results file under docs/qa/<run-id>/. PASS only if exit 0 AND captured evidence (real test output, real uname -a proving Linux) is present (§11.4.69 / §11.4.83).

The image: a small alpine/debian arm64 base extended with tmux build deps (a Containerfile mirroring pkg/crossbuild/linux_container.Containerfile), built once via container build -t <ref> -f Containerfile . and pulled/cached. This is the §11.4.77 regeneration mechanism for the image (gitignored, rebuildable from the tracked Containerfile).

C.3 Anti-bluff test plan

- **L1 pre-build gate** (CM-APPLE-CONTAINER-BACKEND style): apple_container.go exists, implements Backend, capabilities honest (RequiresHostOS=["darwin"], SupportsTargets=[{linux, arm64}]); the runner shells container (not podman/docker); test files exist and go vet/gofmt clean.
- **L3 runtime — Go test (orchestration, fake runner):** apple_container_test.go mirrors linux_container_test.go: happy-path (runCalled, command contains the build token, non-zero ArtifactSize), missing-container-binary honest error, missing-image honest error, zero-byte-is-bluff, capabilities-honest, selector routes Apple-container on NewSelectorForHost("darwin", "arm64") for {linux, arm64}.
- **L3 runtime — REAL shell test (the unforgeable proof):** a Challenge Containers/challenges/scripts/apple_container_linux_challenge.sh that, on a darwin host with container installed:
 1. container system start (idempotent), container image pull <arm64 linux ref>,
 2. boots a real Linux container and container execs **uname -a**, asserting the output contains Linux and aarch64 (proves a REAL Linux kernel ran — an Apple-container micro-VM, not the macOS host whose uname says Darwin),
 3. mounts a host temp dir, writes a sentinel from inside the container, and reads it back on the host (proves the host-dir mount works for getting tmx source in / results out),
 4. **full-flow:** mounts the tmx repo, runs at least one real tmx Linux test (e.g. a single scripts/tests/NN_*.sh) inside, captures its output + exit code into docs/qa/<run-id>/,
 5. SKIP-with-reason (§11.4.81 honest gap) when container is absent OR macOS < 26 makes the chosen kernel unstable — never a faked PASS, never silently degraded.
- **L4 paired mutation (§1.1):** in meta_test_false_positive_proof.sh, mutate the backend so Build returns success without actually invoking the

runner (or strips the `uname Linux` assertion) → the Go test AND the Challenge MUST FAIL. Restore → PASS.

- **§11.4.76 CM-CONTAINERS-USED gate:** assert the `tmx` container-touching code imports `digital.vasic.containers/pkg/crossbuild` (uses the submodule, not a reimplementaion); paired mutation strips the import → gate FAILs. Tracker row records Catalogue-Check: `extend vasic-digital/Containers@<sha>`.
- **Stress/chaos (§11.4.85):** $N \geq 10$ concurrent container boots (each its own micro-VM; assert no deadlock/leak, cleanup in trap EXIT); chaos: container kill mid-build → categorised recovery; first-boot-latency boundary handled with a startup wait, not a fixed sleep.

C.4 What's needed on the host + honest blockers

- **One-time (operator, admin once):** `brew install --cask container` (or the signed `.pkg`); `container system start` (installs default Linux kernel on first run). After that, `container run/exec` are unprivileged (no `sudo`). This is the §11.4.98 one-time-credential/bootstrap exception — outside test execution; the Challenge itself is fully self-driving thereafter.
 - **Honest blocker on THIS host:** macOS 15.5 + container not installed. Apple officially supports macOS 26; on 15.x our flow works within documented limits (we need only host-dir mount + host→guest exec, not inter-container networking), but unreproducible-on-26 bugs may go unfixed. The backend MUST runtime-detect container presence AND macOS version and, when unsupported/unstable, SKIP-with-reason per §11.4.81 and let the existing podman/docker LinuxContainerBackend (Podman Machine on Apple Silicon) serve the same `{linux, arm64}` target — so `tmx` Linux testing on macOS is never blocked outright.
-

Sources verified 2026-06-13

- <https://github.com/apple/container> — README: requirements (Apple Silicon, macOS 26 supported / runs on 15, admin install), `.pkg` install to `/usr/local`, `container system start`, OCI images, lightweight per-container VMs.
- <https://github.com/apple/container/blob/main/docs/command-reference.md> — full CLI flag set for `run/exec/list/stop/kill/delete/system/image/build` incl. `-v/--mount`, `-c/-m/-e/-w/--rm/--name/-d`.
- <https://github.com/apple/container/blob/main/docs/technical-overview.md> — one-VM-per-container via Virtualization framework, OCI compatibility, macOS 15 vmnet networking limitations + 192.168.64.1/24 default CIDR.
- <https://github.com/apple/container/issues/76> — Rosetta-for-Linux status/uncertainty.
- <https://chamodshehanka.medium.com/apples-new-containerization-framework-a-deep-dive-into-macos-s-future-for-developers-cf102643394a> — macOS 15 limitations, Rosetta + kernel 6.13 regressions, macOS 26 networking improvements.
- <https://www.outcoldman.com/blog/2026/05/02/apple-container-tired-of-docker/> — community usage report on Apple Silicon.